

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 - Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 - Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 - JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 - C

```
printf("%d\n", 3 > 2 > 1);
```

5 - Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 - JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

1 – Python: será impresso 4, e depois 512.

2 – Java: será impresso 339, pois 3 é o resultado da primeira soma, depois a string “3” é concatenada, seguida do segundo resultado de soma que é 9.

3 – JavaScript: o resultado é 3 e 52, pois na operação de subtração, a conta com string e inteiro sera permitida, já na segunda operação, a soma irá concatenar a string com o inteiro.

4 – C: imprimirá 0 pois não há nenhum valor definido para %d

5 – Python: imprime a string definida e o resultado booleano da comparação, que é true.

6 – JavaScript: true, pois 0 e “” são vazios, false pois as strings são distintas, e true pois o 0 é considerado igual mesmo com um sendo inteiro e o outro string.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita
 $(3 + 4) * 5 = 35$

da direita para a esquerda
 $3 + (4 * 5) = 23$

com precedência (* antes de +)
 $3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)
 $3 \times 4 + 5 = 27$

Sebesta, 11ª ed. - 57.2 e 57.2.1 - p. 300-305

1 - Quais são as regras de precedência?

As regras de precedência definem a ordem em que operadores com diferentes níveis de prioridade são avaliados em uma expressão. Operadores de maior precedência (como multiplicação $*$ e divisão $/$) são executados antes de operadores de menor precedência e assim por diante.

2 - Quais são as regras de associatividade?

As regras de associatividade definem a ordem de avaliação quando uma expressão contém operadores adjacentes com o mesmo nível de precedência (por exemplo, $A - B + C$). Esquerda para a direita é a mais comum para operadores aritméticos em linguagens como C, Java e Python.

3 - Em que ordem os operandos são avaliados?

Esta questão aborda se as variáveis ou subexpressões que servem de entrada para um operador são processadas da esquerda para a direita ou da direita para a esquerda. A maioria das linguagens modernas não define rigidamente a ordem para permitir otimizações do compilador. No entanto, linguagens como Java garantem estritamente que os operandos sejam avaliados da esquerda para a direita.

4 - Há restrições a efeitos colaterais?

Um efeito colateral funcional ocorre quando uma função chamada dentro de uma expressão altera uma variável global ou um de seus parâmetros

1. *Proibir funções com efeitos colaterais* em expressões
 2. *Definir estritamente a ordem de avaliação dos operandos* (como Java faz), tornando o comportamento previsível, embora ainda possa gerar códigos confusos.
-

5 - O usuário pode sobrecarregar operadores?

A sobrecarga de operadores permite que um único operador (como +) tenha múltiplos significados dependendo do tipo dos seus operandos. Algumas linguagens permitem que o programador defina novos comportamentos para os operadores para tipos criados pelo usuário (como C++ e Python). Outras proíbem a sobrecarga pelo usuário para manter o código mais simples e legível (como Java).

6 - Que mistura de tipos é permitida?

Esta questão trata de expressões de tipo misto (coerção), onde os operandos possuem tipos diferentes (ex: somar um número inteiro com um número de ponto flutuante, como $3 + 4.5$). Conversão implícita (Coerção): O compilador converte automaticamente o tipo mais restrito para o mais amplo (transforma o 3 em 3.0 antes de somar). Design de Linguagem: Linguagens como C/C++ e Java permitem uma ampla gama de coerções implícitas. Já linguagens com tipagem estrita forte, como F# ou Ada, proíbem misturas automáticas, exigindo que o programador faça a conversão de tipo de forma explícita (cast).

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
  x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - §8.1 - p. 328-330

1 - Quantas estruturas de controle são necessárias?

Apenas três estruturas são estritamente necessárias para expressar qualquer algoritmo realizável por um computador. Sendo elas:

1. **Sequência:** Uma instrução executada imediatamente após a outra (ex: atribuições sequenciais).
2. **Seleção:** Escolher um caminho de execução baseado em uma condição (ex: if-else).
3. **Iteração:** Repetir um bloco de código enquanto uma condição lógica for verdadeira (ex: laço while).

Seleção de dois caminhos: o "else pendente"

Regra de Java, C, C++ e C#

O else pertence ao if mais próximo que ainda não tem else. A indentação é ignorada (exceto em Python e F#).

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
```

```
  case 1:
```

```
  case 3: odd += 1;
```

```
  case 2:
```

```
case 4: even += 1;
default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
case SAB, DOM -> "fim de semana";
case SEG, TER, QUA,
QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
i32::MIN..=-1 => "negativo",
0 => "zero",
1..=9 => "um dígito",
_ => "vários dígitos",
}
// sem o _ : não compila
```

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {
int err;
if ((err = verifica_hash()) != 0)
goto fail;
goto fail; // duplicada: desvia SEMPRE
```

```
if ((err = verifica_assinatura()) != 0)
goto fail; // nunca é alcançada
fail:
return err; // 0 = "certificado válido"
}
```

O erro: Note que a linha goto fail; foi duplicada acidentalmente.

O que acontece na prática:

1. Em C, quando um if não usa chaves { }, apenas a primeira linha imediatamente abaixo dele faz parte do bloco condicional.
2. Portanto, o primeiro goto fail; só executa se a verificação do hash falhar (!= 0).
3. Porém, o segundo goto fail; (a linha duplicada) está fora do if. Isso significa que ele executa SEMPRE, independentemente do resultado do teste.

A consequência: O programa pula direto para o rótulo fail:. Com isso, a verificação da assinatura digital (verifica_assinatura()) **nunca é alcançada**. A função retorna err com valor 0, validando um certificado de segurança mesmo que ele fosse falso ou corrompido (uma falha gravíssima que permitia ataques man-in-the-middle).

3. Quem usa o **goto** hoje?

Apesar das críticas, o goto não desapareceu por completo, mas seu uso mudou:

Linguagens que mantêm o **goto**: C, C++, C# e Go. Nessas linguagens, ele ainda é mantido principalmente para manipulação manual de erros e saídas limpas de funções complexas (gerenciamento de memória e liberação de recursos).

Linguagens que removeram: Java não tem o comando **goto**.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) - execute em onecompiler.com

1 – Javascript

O comportamento inesperado ocorre devido a duas regras de projeto do JavaScript:

1. Semântica do `for...in`: Em JavaScript, o laço `for...in` foi projetado para iterar sobre as chaves (propriedades) de um objeto. Quando usado em um array, ele percorre os índices (posições) como strings, e não os valores. Portanto, a variável `p` assume sequencialmente os valores de texto `"0"`, `"1"` e `"2"`.
2. Coerção Implícita de Tipos (Conversão Automática): O operador `+=` realiza a aritmética se ambos os lados forem números. No entanto, quando um dos lados é uma string, o JavaScript converte o outro lado também para string e realiza uma concatenação de texto..

A maneira mais moderna e correta de corrigir isso é substituir o `for...in` pelo laço `for...of`, que foi introduzido especificamente para iterar sobre os valores de uma coleção:

JS index.js × +

```
1  const precos = [10, 20, 30];
2
3  let total = 0;
4
5  for (const p in precos)
6
7    total += p;
8
9  console.log("Total: " + total);
```

Console I/O AI Agent

Total: 0012

